PA 10: Channels

CS 379: Programming Languages (Fall 2026)

Dianelsa M. Chiru M.

Part A: The Specification (50 Points)

Instructions: Answer the following in a separate document (PDF or Markdown).

1. **Bounded Buffer Trace**:

A channel has capacity 2. Thread P (producer) attempts, in order: put(1), put(2), put(3). Thread C (consumer) attempts, in order: get(), get(). The operations are interleaved as: P.put(1), P.put(2), P.put(3) (starts, but the channel is full), C.get(), then P.put(3) completes, then C.get(). Trace the channel's internal buffer state after each completed operation, and identify the exact moment P.put(3) blocks and the exact moment it is allowed to proceed.

Operation	**Buffer State**
Initial	[]
P.put(1)	[1]
P.put(2)	[1, 2]
C.get() → 1	[2]
P.put(3) completes	[2, 3]
C.get() → 2	[3]

put(3) blocks when the buffer reaches capacity 2. It resumes after *C.get()* removes 1 and frees a slot. The final buffer contains [3].

2. **Deadlock by Design:**

Two channels, ch_a and ch_b, both empty and both capacity 1. Thread 1 executes ch_b.put(ch_a.get()). Thread 2 executes ch_a.put(ch_b.get()). Diagram the resulting deadlock — show exactly what each thread is blocked waiting on. Then propose a concrete fix (e.g., an ordering rule, a timeout, or a redesign of who produces vs. consumes on which channel) and explain why it breaks the cycle.



Both threads are blocked waiting for values from empty channels. Neither can execute *put()* because *get()* must finish first.

Fix: Insert an initial value into *ch_a.* Thread 1 can then complete *get()* and send a value to *ch_b,* allowing Thread 2 to continue. This breaks the deadlock.

3. **Bounded vs. Unbounded, and the GC Connection:**

An unbounded channel's put never blocks. Explain, referencing PA 9's GCHeap, why an unbounded channel can still cause a memory problem even though no individual object is leaked or uncollected — connect this to reference counting specifically (what is holding the refcount up while a value sits in the buffer?).

A bounded channel limits stored values, while an unbounded channel allows unlimited values without blocking *put().*

In PA 9, *GCHeap.incref()* increases the reference count when the channel stores a handle. The channel keeps that reference while the value remains buffered.

If producers are faster than consumers, an unbounded buffer keeps growing, consuming memory even without a traditional memory leak. Memory is released when consumers receive the values and call *decref(),* assuming no other references remain.